

IT Ticket Manager – Yazılım Tasarım Dokümanı

Hazırlayan: İsmail Fatih Çolak

Yıl: 2025 / 2026

Sürüm: v8 : 06/Haziran/2026

Kuruluş: 32Bit Ofisi için Hazırlanmıştır

Referanslar: ITIL v4, ServiceNow, Jira Service Management

SAU Mail: ismail.colak2@ogr.sakarya.edu.tr

TASARIM DOKÜMANI İÇERİĞİ

- 0. DOKÜMAN YAKLAŞIMI VE BAĞLAYICILIK – 3**
- 1. DOMAIN MODEL AYRIMI – 5**
- 2. INCIDENT VE SERVICE REQUEST TASARIMI – 7**
- 3. ÖNCELİKLENDİRME (URGENCY / IMPACT) – 7**
- 4. TICKET LIFECYCLE & EDGE CASE'LER – 8**
- 5. SLA, ESCALATION VE RİSK YÖNETİMİ – 9**
- 6. TIMELINE, INTERNAL/EXTERNAL İÇERİK – 11**
- 7. ATTACHMENT YÖNETİMİ – 12**
- 8. FAILURE MODES – 12**
- 9. RBAC VE YETKİ MATRİSİ – 13**
- 10.EVENT CATALOG - 13**
- 11.API & UI CONTRACT – 15**
- 12.REPORTING – 18**
- 13.NON-FUNCTIONAL REQUIREMENTS - 19**
- 14.TRACEABILITY TABLOSU - 19**
- 15.TEKNOLOJİ STACK'I VE KULLANILAN YAZILIMLAR - 20**
- 16.SONUÇ - 22**

0. Doküman Yaklaşımı ve Bağlayıcılık

Bu doküman, Yazılım Analiz Dokümanı'nda tanımlanan iş ihtiyaçları ve süreç beklentilerinin teknik tasarım düzeyindeki doğrudan karşılığı olarak hazırlanmıştır. Analiz dokümanı sistemin ne yapması gerektiğini tanımlar; bu tasarım dokümanı ise bu beklentileri yorum katmadan, uygulanabilir teknik kurallara dönüştürür.

Bu kapsamda:

- Tasarım, analiz dokümanında tanımlı olmayan yeni iş davranışları üretmez
- Teknik kararlar, analiz kapsamını daraltamaz veya genişletemez
- Tüm süreç, rol ve kural tanımları analiz dokümanındaki ilkelere dayanır

Bu doküman yaşayan bir tasarım dokümanıdır. Yapılacak revizyonlar, analiz dokümanında tanımlanan hedefler korunarak gerçekleştirilir.

Application Layer & Orchestration

Bu doküman, domain kurallarını ve izin verilen davranışları tanımlar. Bu davranışların hangi sırayla tetiklendiği ve use-case bazlı orkestrasyonu, Application Layer sorumluluğundadır.

MVP fazında Application Layer;

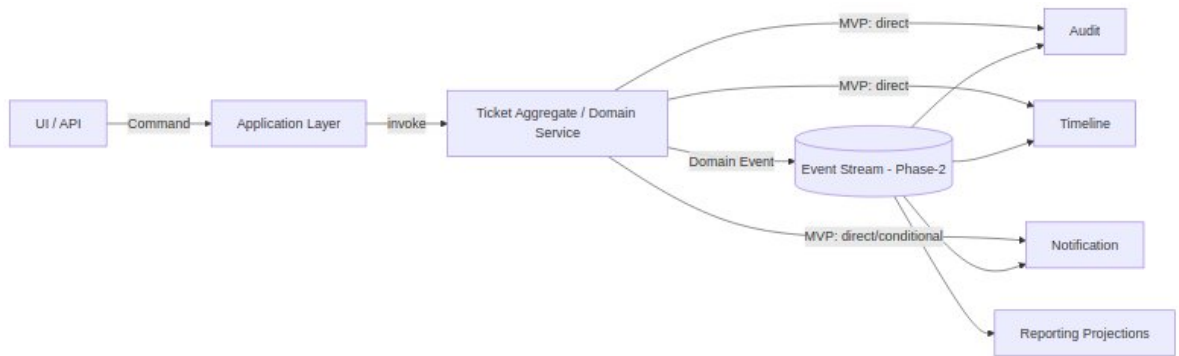
- Domain kurallarını ihlal etmeyecek şekilde çağrılarını koordine eder
- SLA clock, status transition ve audit tetiklerini senkronize eder
- UI ve dış sistemlerden gelen talepler için tek giriş noktası sağlar

Bu katmanın detaylı tasarımı, mimari doküman kapsamında ele alınacaktır.

Command → Domain → Event Ayrımı

- **Command:** Application Layer tarafından başlatılan use-case isteğidir (örn. ChangeStatusCommand).
- **Domain:** Aggregate/Domain Service kuralları uygular, invariant'ları korur.
- **Domain Event:** Domain içinde gerçekleşen önemli değişimi bildirir; audit/timeline/notification gibi supporting süreçleri besler.

MVP'de supporting işlemler (audit/timeline/notification) doğrudan transaction içinde tetiklenebilir. Phase-2'de ölçek ihtiyacı oluştuğunda event-driven akışa geçiş hedeflenir.



Sorumluluk Ayrımı (Netleştirme)

- **Application Layer:**
 - Use-case bazlı akışları orkestre eder
 - Status transition, SLA trigger ve escalation çağrı sırasını belirler
 - Domain kurallarını *uygulamaz*, yalnızca çağırır
- **Domain (Aggregate + Domain Service):**
 - Status değişiminin geçerli olup olmadığını doğrular
 - SLAClock state geçişlerini kontrol eder
 - Escalation kararlarının kurallara uygunluğunu garanti eder

Bu ayırım ile:

- Akış kontrolü Application Layer'da
- İş kuralı doğrulaması Domain'dedir

Phase Boundary (Bilinçli Kapsam Sınırı)

Bu SDD, analiz dokümanındaki iş davranışlarını teknik kurallara dönüştürmeye odaklanır.

Bu fazda aşağıdaki mimari ayrıntılar *bilinçli olarak* detaylandırılmamıştır:

- Application layer (use-case orchestration)
- Domain service vs aggregate sınırları
- Command / Query ayrımı (CQRS)

Phase-2 Öncelikli Konular

1. Event granularity ayrıştırma
2. Çoklu SLA clock desteği
3. Notification motoru genişletme
4. CQRS değerlendirmesi
5. Reporting optimizasyonu

Gerekçe: MVP'de hedef; statü akışları, SLA clock, RBAC, audit/timeline izlenebilirliği gibi iş davranışlarının doğrulanmasıdır.

Bu mimari kararlar, ilk çalışma prototipinden elde edilecek geri bildirimlere göre Phase-2'de kesinleştirilecektir.

1. Domain Model Ayrımı

Sistem, analiz dokümanında tanımlanan iş süreçlerini net ve kontrol edilebilir şekilde modelleyebilmek amacıyla core ve supporting domain'ler olarak ayrıştırılmıştır.

1.1 Core Domain

Core domain, doğrudan iş değeri üreten ve analiz dokümanında tanımlanan temel süreçleri kapsar:

- Ticket
- Incident
- Service Request
- Önceliklendirme (Urgency / Impact)
- SLA & Escalation

Core Domain - Aggregate ve Invariant'lar

Aggregate Root	Kapsam	Temel Invariant
Ticket	Status, Priority, SLAClock	Geçersiz status transition yapılamaz
Ticket	SLAClock	SLA state yalnızca tanımlı akışta ilerler
Ticket	Assignment	Yetkisiz rol assignment yapamaz
ServiceRequestTicket	Approval	Onay tamamlanmadan RESOLVED olamaz

Bu alanlar, sistemin iş perspektifinden beklenen ana davranışlarını belirler ve süreç akışını doğrudan etkiler.

Domain Service vs Aggregate Ayrımı (Tasarım Prensipleri)

Bu sistemde domain modeli, davranışların nerede konumlanacağına dair aşağıdaki prensiplerle tasarlanmıştır:

- **Aggregate Root (Ticket):**
 - State (status, priority) tutarlılığından sorumludur
 - Geçersiz status transition'ları engeller
 - SLAClock state geçişlerini doğrular
 - Invariant'ları her zaman kendi sınırları içinde korur
- **Domain Service:**
 - Birden fazla aggregate veya dış veri gerektiren kararları içerir
 - Priority hesaplama (Urgency / Impact matrisi)
 - SLA escalation kararları (%70 / %85 / %100)
 - Approval sonucu değerlendirme (Service Request)

Bu ayırım ile:

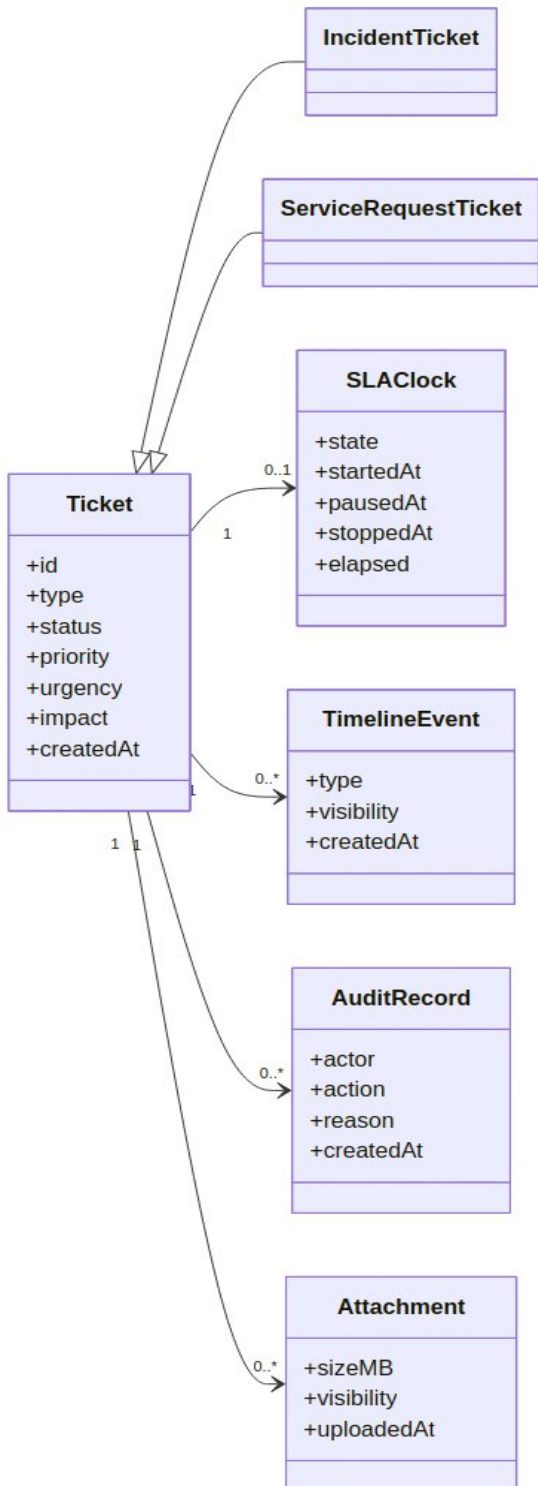
- Aggregate'lar kuralları **uygular**
- Domain Service'ler kararları **hesaplar**
- Application Layer ise yalnızca **orkestrasyon** yapar

1.2 Supporting Domain

Supporting domain, core domain süreçlerinin izlenebilirliğini, denetlenebilirliğini ve raporlanabilirliğini sağlamak amacıyla konumlandırılmıştır:

- Audit
- Logging
- Timeline
- Notification
- Reporting

Supporting domain bileşenleri, core domain davranışlarını tetiklemez veya değiştirmez; yalnızca gerçekleşen aksiyonları kaydeder ve görünür kılar. Bu ayırım, analiz dokümanında vurgulanan kurallara dayalı süreç, şeffaflık ve denetlenebilirlik ilkelerinin teknik karşılığını oluşturur.



Aggregate Boundary – Ölçeklenebilirlik Notu

Attachment, TimelineEvent ve AuditRecord bileşenleri MVP fazında Ticket aggregate içinde modellenmiştir.

Sistem ölçeklendikçe bu bileşenler:

- Ayrı aggregate
- Ayrı bounded context olarak ayrıştırılmaya uygundur.

Mevcut tasarım, MVP için tutarlılığı ve basitliği önceliklendirir.

Split Sinyalleri (Ölçülebilir)

Aşağıdaki sinyaller gözlemlenirse Attachment / Timeline / Audit bileşenleri Ticket aggregate'tan ayrıştırılmalıdır:

- **Performans:** Ticket update işleminde P95 latency > 300ms (yük altında)
- **Veri büyümesi:** Tek bir ticket için timeline event sayısı > 500 veya attachment > 50
- **Çakışma:** Aynı ticket üzerinde optimistic lock conflict oranı > %5
- **Depolama:** Ticket satırı veya ilişkili join maliyeti belirgin artış (örn. sık timeout)
- **Organizasyonel:** Timeline/Audit farklı ekip tarafından ayrı sprintlerle geliştirilmeye başlanır

Bu sinyaller, Phase-2'de bounded context ayrıştırma kararının tetikleyicileri olarak kullanılır.

2. Incident ve Service Request Tasarımı

2.1 Ayrımın Nedeni

Analiz dokümanında Incident ve Service Request süreçlerinin **karışması operasyonel risk** olarak tanımlanmıştır.

2.2 Domain Ayrımı

Ticket üst varlıktır; alt türler şunlardır:

- IncidentTicket
- ServiceRequestTicket

2.3 Incident Kuralları

- Acil durum odaklı
- SLA zorunlu
- Otomatik escalation açık

2.4 Service Request Kuralları

- Katalog bazlı
- SLA zorunlu
- Onay (approval) adımı eklenebilir

Approval Flow (Service Request)

- Service Request, opsiyonel onay adımı içerebilir
- Approval tamamlanmadan RESOLVED durumuna geçilemez
- Reddedilen talepler CLOSED durumuna alınır (reason + audit)

Approval akışı, ticket yaşam döngüsünü genişletmez; mevcut statüler içinde kontrol mekanizması olarak çalışır.

Tüm ticket türleri tekil bir SLAClock üzerinden takip edilir. Service Request katalog kalemlerine bağlı farklı SLA türlerinin ve response/resolution clock ayrımının desteklenmesi 5.1.1 Extensibility kapsamındadır.

3. Önceliklendirme (Urgency / Impact)

3.1 Priority Modeli

Priority aşağıdaki matrisle hesaplanır:

Priority = f(Urgency, Impact)

Impact \ Urgency	Low	Medium	High
Low	LOW	LOW	MEDIUM
Medium	LOW	MEDIUM	HIGH
High	MEDIUM	HIGH	CRITICAL

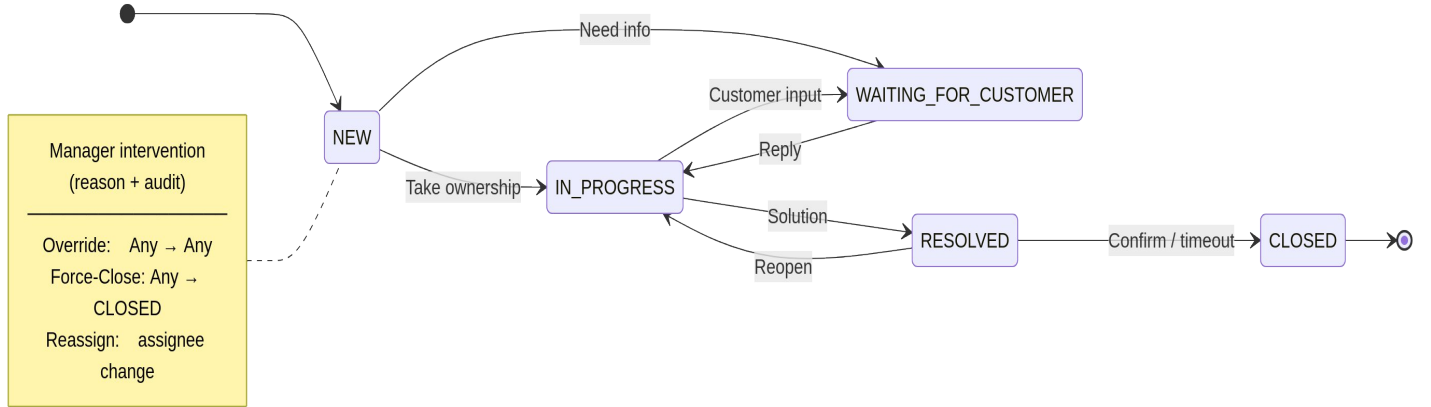
3.2 Değişim Kuralları

- Customer urgency önerir
- Agent impact belirler
- Priority değişimi **audit zorunludur**

4. Ticket Lifecycle & Edge Case'ler

4.1 Temel Lifecycle

NEW → IN_PROGRESS → WAITING_FOR_CUSTOMER ↔ IN_PROGRESS → RESOLVED → CLOSED



4.2 Yanıtsız Müşteri (Abandoned Case)

WAITING_FOR_CUSTOMER state'inde geçirilen süre, SLAClock.pausedAt referans alınarak ölçülür. Periyodik bir tarayıcı (AbandonedTicketScheduler) iki eşiği değerlendirir:

Aşama	Eşik	Aksiyon	Audit	Timeline	Bildirim
Hatırlatma	reminderThreshold	Customer'a mail	AUTO_REMINDER_SENT	System Event	-
Terk İşareti	abandonedThreshold	Manager kuyruğu	AUTO_TIMEOUT_FLAGGED	System Event	Her Zaman

Otomatik kapatma yapılmaz. Terk işareti yalnızca yönetsel müdahaleyi tetikler; nihai kapatma Manager tarafından gerekçeli forceClose ile uygulanır. Bu ayırım, 0.2.4 "Rol Bazlı Yetkilendirme" ilkesini bozmadan abandoned akışını süreç içine dahil eder.

Idempotency. Ticket aggregate'i reminderSentAt ve abandonedFlaggedAt zaman damgalarını tutar. Aynı ticket için hatırlatma ve terk işareti birden fazla kez üretilmez. Ticket WAITING_FOR_CUSTOMER state'inden çıktığında her iki zaman damgası temizlenir; sonraki bekleme periyodu temiz başlar.

Konfigürasyon Parametreleri:

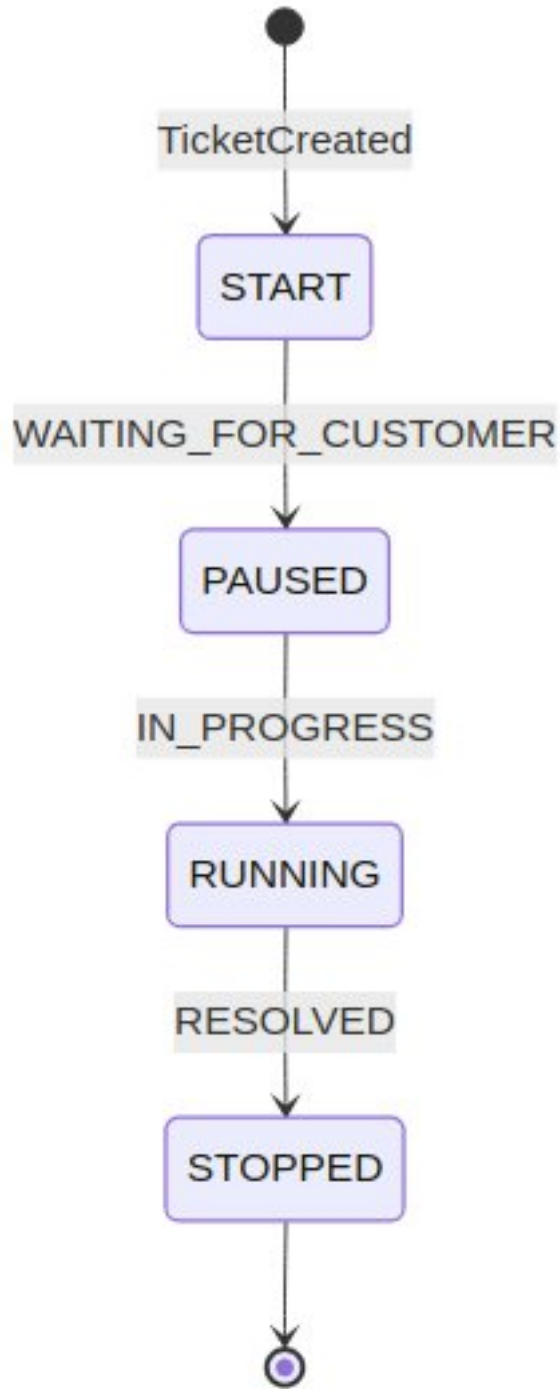
Parametre	Varsayılan	Açıklama
itsm.abandoned.reminder-hours	24	Hatırlatma Eşiği
itsm.abandoned.timeout-hours	168	Terk Eşiği
itsm.abandoned.scan-cron	0 0 * * * *	Tarama Sıklığı
Itsm.abandoned.from-address	noreply@itsm.local	Bildirim Gönderici Adresi

5. SLA, Escalation ve Risk Yönetimi

5.1 SLA Clock

- TicketCreated → START
- WAITING_FOR_CUSTOMER → PAUSE
- IN_PROGRESS → RESUME
- RESOLVED → STOP

5.1.1 SLA Clock State Flow



Bu akış, analiz dokümanında tanımlanan “SLA’nın adil yönetimi” ilkesinin teknik karşılığını temsil eder.

Extensibility Notu

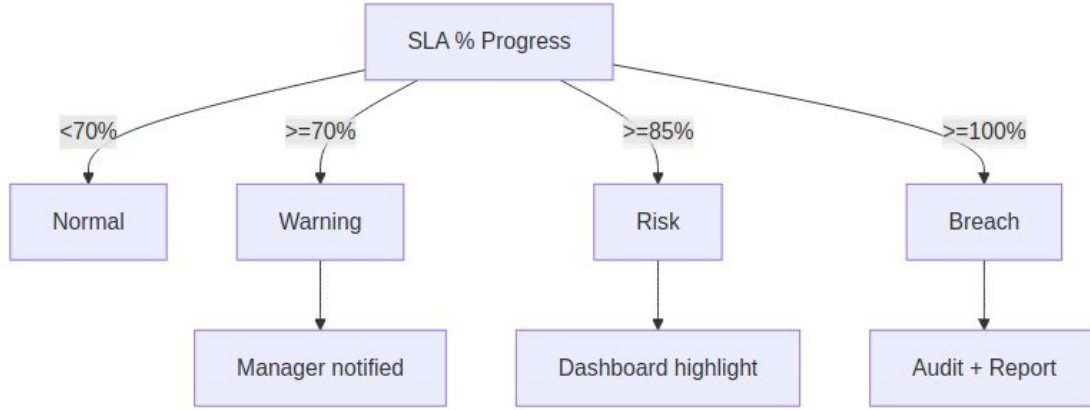
MVP fazında sistem tek bir SLAClock üzerinden ilerler.

Phase-2’de:

- Response SLA
- Resolution SLA gibi farklı SLA tipleri tanımlanması durumunda, her SLA türü için ayrı clock modeli uygulanabilir.

5.2 Escalation Seviyeleri

SLA Seviyesi	Tetik Noktası	Sistem Davranışı
Normal	< %70	Normal süreç devam eder
Warning	≥ %70	Manager bilgilendirilir
Risk	≥ %85	Dashboard üzerinde vurgulanır
Breach	≥ %100	Audit kaydı + raporlama



6. Timeline, Internal / External İçerik

6.1 İçerik Türleri

- External Comment
- Internal Comment
- External Worklog
- Internal Worklog
- System Event
- Service Quality Complaint

İçerik Türü	Customer Görür	Agent Görür	Manager Görür
External Comment	✓	✓	✓
Internal Comment	✗	✓	✓
External Worklog	✓	✓	✓
Internal Worklog	✗	✓	✓
System Event	✓	✓	✓
Service Quality C.	✓	✓	✓

6.2 Timeline Davranışı

- Customer yalnızca external kayıtları görür
- Agent/Manager tümünü görür
- Her kayıt timeline event üretir.

7. Attachment Yönetimi

Kural	Açıklama
Görünürlük	Internal / External olarak ayrılır
Maksimum Boyut	10 MB
Dosya Türü	Sistem tarafından kısıtlanabilir
Yetkilendirme	Rol bazlı upload
Denetim	Her upload audit + timeline event üretir

Her attachment işlemi audit ve timeline event üretir.

8. Failure Modes

Bu bölüm, analiz dokümanındaki “kurallara dayalı süreç” ve “denetlenebilirlik” ilkelerinin hata/istisna anlarındaki teknik karşılığını tanımlar.

Failure Mode	Örnek Durum	Sistem Tepkisi	HTTP	Audit / Timeline
Illegal Status Transition	CLOSED → IN_PROGRESS denenmesi	İşlem reddedilir, mevcut state korunur	409 Conflict	Audit: ✓ (deneme kaydı), Timeline: opsiyonel
Version Conflict	İki ajan aynı ticket’ı eş zamanlı günceller	İlk commit kazanır, sonraki reddedilir	409 Conflict	Audit: ✗ Opsiyonel, Timeline: ✗
SLA Clock Inconsistency	STOPPED iken RESUME tetiklenmesi	SLA state normalize edilir veya işlem reddedilir	409 Conflict	Audit: ✓, Timeline: System Event
Unauthorized Override Attempt	Agent “ManagerOverride” dener	Yetkisiz işlem reddedilir	403 Forbidden	Audit: ✓ (security), Timeline: ✗
Invalid Payload	Priority değeri tanımsız	Validasyon hatası döner	400 Bad Request	Audit: ✗ (opsiyonel), Timeline: ✗
Not Found	TicketId yok	Bulunamadı	404 Not Found	Audit: opsiyonel, Timeline: ✗

Timeline Kuralı

Security ve yetki ihlali kaynaklı failure’lar customer timeline’a yansımaz.

Bu olaylar yalnızca audit kayıtlarında tutulur.

9. RBAC ve Yetki Matrisi

Aksiyon	Customer	Agent	Manager
Ticket Oluşturma	✓	✓	✓
Ticket Durumu Değiştirme	✗	✓	✓ (override)
Ticket Atama	✗	✓	✓
Öncelik Değiştirme	✗	✓	✓
Ticket Kapama	✗	✓	✓
Yönetici Müdahalesi	✗	✗	✓ (reason + audit)

Manager override işlemleri **reason + audit zorunlu**.

10. Event Catalog

Event Granularity Prensibi

MVP fazında event'ler, operasyonel izlenebilirliği sağlamak amacıyla daha genel isimlendirilmiştir (örn. TicketUpdated).

Phase-2'de event-driven mimariye geçişte:

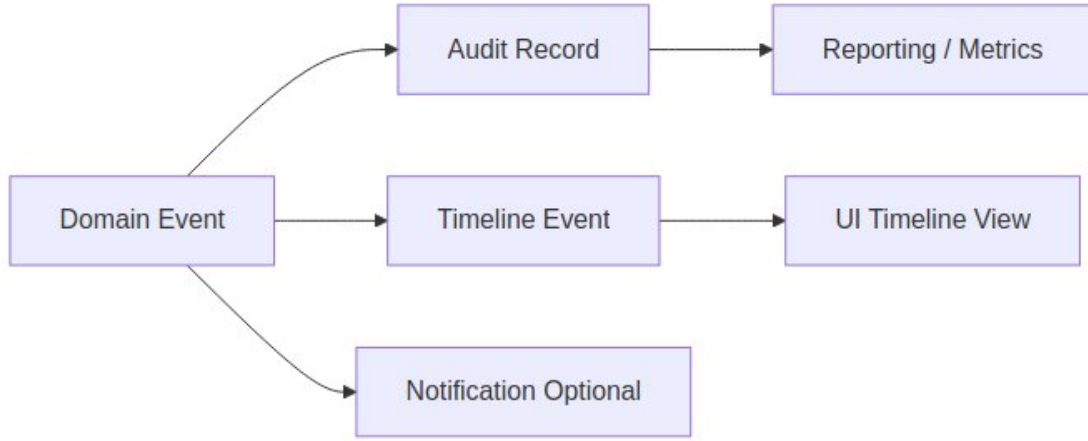
- AssignmentChanged
- ImpactChanged
- UrgencyChanged
- TicketCreated
- TicketUpdated
- StatusChanged
- PriorityChanged
- SLAPaused
- SLAResumed
- SLABreachRisk
- SLABreached
- ManagerOverride
- AttachmentAdded
- Service Quality Complaint
- Customer Feedback

Event	Audit Kaydı	Timeline Event	Notification
TicketCreated	✓	System Event	Opsiyonel
StatusChanged	✓	System Event	Opsiyonel
PriorityChanged	✓	System Event	Opsiyonel
SLAPaused	✓	System Event	✗
SLAResumed	✓	System Event	✗
SLABreachRisk	✓	System Event	✓
SLABreached	✓	System Event	✓
ManagerOverride	✓ (reason zorunlu)	System Event	Opsiyonel
AttachmentAdded	✓	Attachment Event	Opsiyonel
SERVICE_QUALITY_COMPLAINT	✓	Timeline Event	✓
CUSTOMER_FEEDBACK	✓	Notification Event	Opsiyonel
ServiceQualityComplaint	✓	External Commit	✓
CustomerFeedback	✓	System Event	Opsiyonel
AutoReminderSent	✓ (System actor)	System Event	✗
AutoTimeoutFlagged	✓ (System actor)	System Event	✓

Her event, denetlenebilirlik ve geriye dönük inceleme amacıyla audit ve timeline ile ilişkilendirilir.

API Response Edge-Cases (Özet)

Senaryo	HTTP	Açıklama
Illegal status transition	409 Conflict	Kurallara aykırı geçiş reddedilir
Unauthorized action	403 Forbidden	RBAC ihlali
Ticket not found	404 Not Found	Geçersiz id
Invalid payload	400 Bad Request	Validasyon hatası
Concurrency conflict	409 Conflict	Versiyon uyumsuzluğu
Version Conflict	409 Conflict	Eş Zamanlı güncelleme sırasında versiyon uyumsuzluğu



Notification Stratejisi

Event	Notification Davranışı	Gerekçe
TicketCreated	Koşullu	Assignment varsa bilgilendirme
StatusChanged	Koşullu	Customer-facing statülerde
SLABreachRisk	Her zaman	Proaktif müdahale
SLABreached	Her zaman	Operasyonel kritik
ManagerOverride	Opsiyonel	Audit öncelikli
AttachmentAdded	Koşullu	External attachment durumunda
ServiceQualityComplaint	Her Zaman	Yönetmel müdahale kanalı
AutoTimeoutFlagged	Her Zaman	Terk işareti yönetim onayı gerektirir
CustomerFeedback	Opsiyonel	Bilgilendirme amaçlı

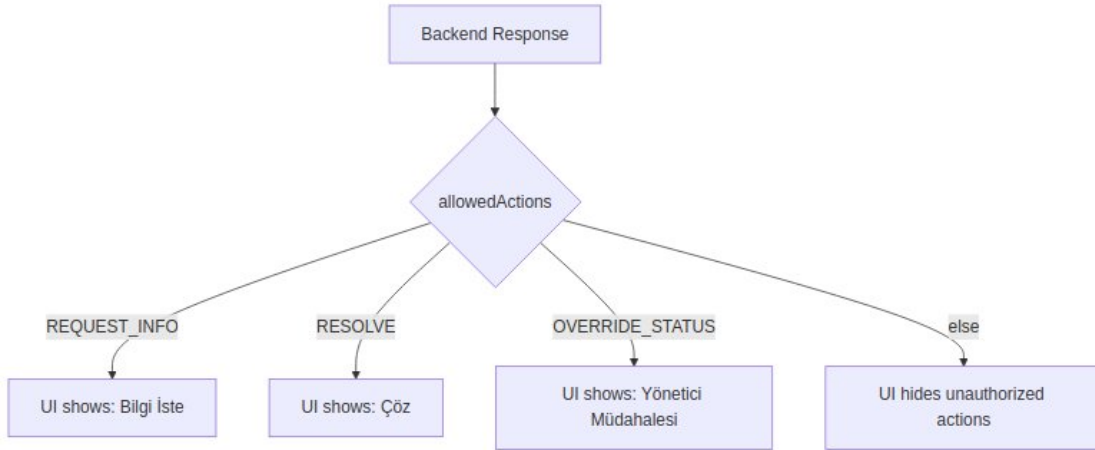
Notification mekanizması, kullanıcıyı bilgilendirmeyi amaçlar; audit ve timeline kayıtlarının yerini almaz.

Notification kuralları aşağıdaki prensiplere göre çalışır:

- SLA bazlı: SLABreachRisk, SLABreached → her zaman
- Status bazlı: Customer-facing status değişimleri
- Rol bazlı: Manager override yalnızca yöneticilere

11. API & UI Contract

Backend her ticket response'unda:



```
{
  "status": "IN_PROGRESS",
  "priority": "HIGH",
  "allowedActions": ["REQUEST_INFO", "RESOLVE"],
  "slaRiskLevel": "WARNING"
}
```

Ticket Status	Rol	allowedActions Örnekleri
NEW	Agent	TAKE_OWNERSHIP, REQUEST_INFO
IN_PROGRESS	Agent	REQUEST_INFO, ADD_WORKLOG, RESOLVE
WAITING_FOR_CUSTOMER	Customer	ADD_COMMENT, ADD_ATTACHMENT
RESOLVED	Customer	CONFIRM_CLOSE, REOPEN_REQUEST
ANY	Manager	OVERRIDE_STATUS, REASSIGN, CHANGE_PRIORITY

API Failure Response Örnekleri

JSON formatında örnek Failure Response'ları aşağıda verilmiştir:

// 409 - Illegal Status Transition

```
{ "errorCode": "INVALID_STATUS_TRANSITION",
  "message": "Ticket cannot transition from CLOSED to IN_PROGRESS", "currentStatus": "CLOSED"
}
```

// 403 - Unauthorized Action {

```
"errorCode": "UNAUTHORIZED_ACTION",
"message": "User role is not allowed to perform this action"
}
```

Reopen Senaryosu

- REOPEN_REQUEST aksiyonu yalnızca RESOLVED durumunda kullanılabilir
- Ticket, IN_PROGRESS durumuna geri döner
- SLAClock:
 - Daha önce STOPPED ise yeniden başlatılır
 - Önceki SLA süresi audit kayıtları üzerinden korunur

Endpoint Sözleşmesi

Endpoint kataloğu, request/response şemaları, hata kodları ve örnek payload'lar ayrı bir referans dokümanda toplanmıştır: docs/api-ui-contract.md. Bu bölümde tanımlanan sözleşme

İlkeleri:

- Tüm endpoint'ler /api/v1 base path'i altında yer alır (path versioning).
- Kimlik doğrulama Bearer JWT ile sağlanır; tokenlar Keycloak realm üzerinden alınır.
- Listeleme uçları sayfalanır (page, size) ve sayfa büyüklüğü için üst sınır uygulanır.
- Hata yanıtları { "errorCode", "message" } formatındadır; mesaj alanı i18n catalog üzerinden çözülür.
- TicketResponse.allowedActions UI tarafının buton yetkilendirmesinde kullandığı sözleşme alanıdır; bu alan dışında UI yetkilendirme kararı vermez.

Concurrency & Tutarlılık

Eşzamanlı talepler sonucunda oluşabilecek tutarsızlıklar, optimistic locking ve transaction bazlı güncellemelerle yönetilir.

- Status değişimleri atomik olarak ele alınır
- SLAClock state güncellemeleri tekil işlem kapsamında yapılır
- Ticket aggregate, versiyonlanan tek tutarlılık sınırındadır

Çakışma Senaryoları

Senaryo	Çözüm
Status change + SLA update çakışması	Aynı transaction, tek version
Agent update + Manager override	Manager override öncelikli
İki agent aynı anda update	Versiyon uyumsuzluğu → 409
Reopen + resolve çakışması	İlk commit kazanır

Kilitleme ve Versiyonlama Stratejisi

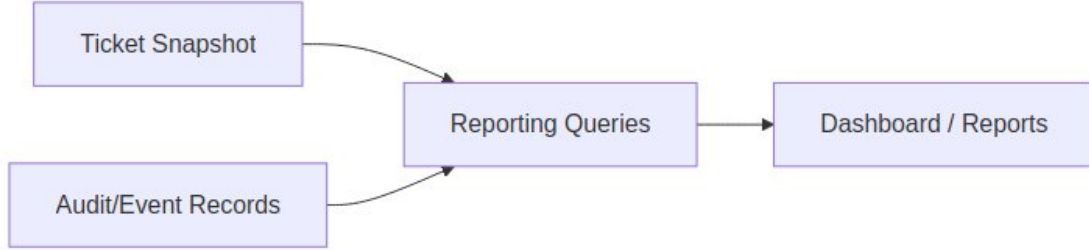
- Ticket üzerinde **version** alanı bulunur
- Status, assignment ve SLAClock güncellemeleri aynı transaction içinde yapılır
- Versiyon uyumsuzluğunda işlem reddedilir
- HTTP 409 Conflict döndürülür

Bu yaklaşım, aynı ticket üzerinde eşzamanlı işlem denemelerinde veri tutarlılığını garanti eder.

11.1 Reporting Veri Kaynağı

MVP fazında raporlama; full event-sourcing yaklaşımı yerine, ticket snapshot verileri ve event/audit kayıtlarından türetilen ölçümlerle üretilir.

- **Snapshot kaynakları:** Ticket.status, Ticket.priority, SLAClock.elapsed, timestamps
- **Event/Audit kaynakları:** StatusChanged, PriorityChanged, SLABreachRisk, SLABreached, ManagerOverride



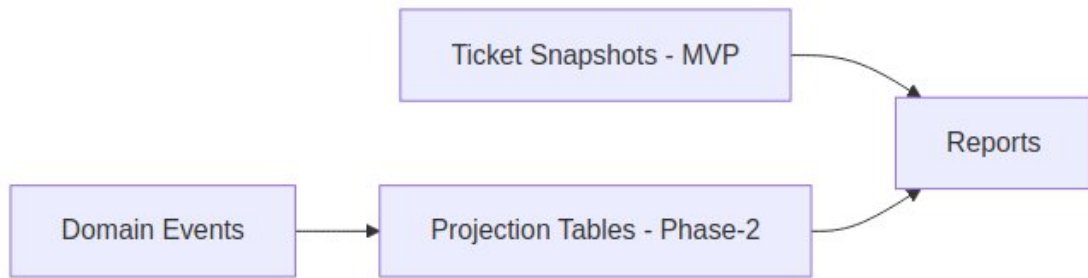
Snapshot Yaklaşımının Sınırı (Ne zaman yetmeyecek?)

Snapshot tabanlı raporlama, MVP için yeterlidir; ancak aşağıdaki durumlarda yetersiz kalır:

- **Yük:** Rapor sorguları toplam okuma trafiğinin > %30'una ulaştığında
- **Gecikme:** Dashboard metriği “near real-time” (örn. < 5 sn) beklenmeye başladığında
- **Karmaşıklık:** “Zaman içinde trend / adım adım akış” (funnel) gibi event-temelli analizler gerektiğinde
- **Maliyet:** Sık rapor sorguları üretim veritabanını baskıladığında (P95 query > 500ms)

Phase-2 çözüm yönü:

- Domain event'lerden türetilen **read model / projection** tablosu oluşturmak
- Raporları bu projection üzerinden almak (operational DB yükünü azaltmak)



12. Reporting

Metrik Tanımları

Metrik	Hesaplama
Ortalama çözüm süresi	IN_PROGRESS → RESOLVED
SLA compliance	SLAClock.elapsed ≤ SLA threshold
Escalation sayısı	SLABreachRisk + SLABreached

Minimum raporlar:

- SLA compliance
- Ortalama çözüm süresi
- Escalation sayıları
- Agent iş yükü

Feedback Metrikleri

Müşteri geri bildirimleri (4.4.6, Analiz Dokümanı), kapatılan ticket'lar üzerinden toplanır ve yönetim raporu olarak sunulur.

Metrik	Hesaplama
Toplam Geri Bildirim	COUNT (TicketFeedback)
Ortalama Puan	AVG (rating)
Puan Dağılımı	1-5 yıldız bazında sayım
Agent Kırılımı	Geri bildirim anındaki assignee bazında count + ortalama

Agent kırılımındaki snapshot, geri bildirimin verildiği andaki sorumlu Agent'ı sabitler; sonraki yeniden atamalar geçmiş ölçümleri etkilemez.

Agent İş Yüğü

Ekip yükü ve dağılım dengesi, agent başına açık ticket sayısı ve status dağılımı üzerinden ölçülür.

Metrik	Hesaplama
Toplam Yük	Agent başına CLOSED hariç ticket sayısı
Status dağılımı	(NEW, IN_PROGRESS, WAITING_FOR_CUSTOMER, RESOLVED) kırılımı

Rapor toplam yüke göre azalan sırada sunulur.

13. Non-Functional Requirements

- i18n: message catalog tabanlı
- Performance: pagination zorunlu
- Security: JWT + RBAC
- Observability: log + audit + timeline
- Pagination: max page size = 50
- JWT expiry: 15 dakika
- Audit retention: minimum 1 yıl

Not, Aşağıdaki konular bu fazda tasarım kapsamında ele alınmamıştır:

- High availability ve disaster recovery senaryoları
- Otomatik ölçekleme ve load balancing
- Harici observability platformları (APM, tracing)

Bu başlıklar, sistem davranışı doğrulandıktan sonra ileri fazlarda değerlendirilir.

Sınırlar ve Politikalar:

- Listeleme uçlarında sayfa büyüklüğü sunucu tarafında üst sınıra çekilir; istemcinin daha büyük sayfa talep etmesi sınıra düşürülerek karşılanır.
- Access token ömrü 15 dakika ile sınırlıdır; yenileme refresh token üzerinden yapılır.
- Audit kayıtları kalıcıdır; geçmişe dönük denetim kayıplarını önlemek için silme işlemi tanımlanmamıştır. Saklama süresi politikası kurumsal düzenleme doğrultusunda Phase-2'de uygulanacak retention katmanı ile zorlanır.

14. Analiz → Tasarım İzlenebilirlik (Traceability)

Analiz Beklentisi	Tasarım Bölümü	Kanıt Artefakt
Süreçlerin kurallı işlemesi	Bölüm 4	Status transition kuralları
SLA'nın adil yönetimi	Bölüm 5	SLA clock + risk seviyeleri
Önceliklendirme şeffaflığı	Bölüm 3	Impact / Urgency matrisi
Denetlenebilirlik	Bölüm 6–9	Audit + timeline + event catalog
Yönetici müdahalesi	Bölüm 8	Override + reason zorunluluğu
UI davranışlarının kontrollü olması	Bölüm 10	allowedActions kontratı
Müşteri Geri Bildirimi	Bölüm 12	TicketFeedback aggregate+reporting endpoint
Yanıtsız Talep Akışı	Bölüm 4.2	AbandonedTicketScheduler + idempotency alanları
Servis Kalitesi Şikayeti	Bölüm 6.1, 10	ServiceQualityComplaint event tipi

Glossary

Terim	Açıklama
Manager Override	Yönetici tarafından yapılan istisnai müdahale
AllowedActions	UI'da gösterilecek izinli aksiyonlar
SLAClock	SLA süresini takip eden mekanizma
Abandoned Ticket	Tanımlanmış sürede WAITING_FOR_CUSTOMER durumunda kalan ticket
Service Quality Complaint	Müşterinin süreç kalitesine yönelik açtığı, EXTERNAL görüntülenebilirlik
Customer Feedback	Kapatılan ticket için müşteri tarafından sağlanan geri dönüş
Overtime Ticket	SLA süresini aşmış ve henüz çözüme veya kapatmaya alınmamış ticket

15. Teknoloji Stack'i ve Kullanılan Yazılımlar

Bu bölüm, sistemin uçtan uca işletilmesi için kullanılan tüm bileşenleri ve sürümlerini listeler. Bileşenler, mimari katmanlara göre gruplanmıştır.

15.1. FRONTEND

Bileşen	Sürüm	Amaç
React	18.3.1	UI Çerçevesi
Vite	7.3.1	Build / Dev sunucusu
TypeScript	5.6.3	Statik tip kontrolü
Axios	1.7.9	HTTP istemcisi
Keycloak JS Adapter	26.0.7	OIDC akışı, token yönetimi
I18next + react-i18next	24.2 / 15.2	Çok dilli mesaj kataloğu
@fontsource-variable/inter	5.2.8	Tipografi

15.2. BACKEND

Bileşen	Sürüm	Amaç
Java	21	Temel Yazılım
Spring Boot	3.3.5	Uygulama çatısı
Spring Web	(BOM)	REST API
Spring Security	(BOM)	Kimlik doğrulama, yetkilendirme
Spring OAuth2 Resource Server	(BOM)	JWT doğrulama
Spring Validation	(BOM)	İstek doğrulama (Jakarta Validation)
Spring Actuator	(BOM)	Sağlık ve metrik uçları
Spring Mail	(BOM)	Müşteri hatırlatma e-postaları

15.3. VERİTABANI VE ORM

Bileşen	Sürüm	Amaç
PostgreSQL	16	İlişkisel veri katmanı
Spring Data JPA / Hibernate	(BOM)	ORM
Flyway	(BOM)	Şema versiyonlaması (V1-V4)
Caffeine	(BOM)	Yerel önbellek (rapor sonuçları)

15.4. KİMLİK VE YETKİ

Bileşen	Sürüm	Amaç
Keycloak	26.0	Realm yönetimi, JWT üretimi
Keycloak TOTP	—	İki faktörlü kimlik doğrulama

15.5. İŞ AKIŞI

Bileşen	Sürüm	Amaç
Kogito (new-gen jBPM)	10.1.0	BPMN süreç motoru
jBPM Spring Boot Starter	(BOM)	Ticket lifecycle orchestration

15.6. MESAJLAŞMA VE LOG PIPELINE

Bileşen	Sürüm	Amaç
Apache Kafka	3.7.2	Log event taşıma
Spring Kafka	(BOM)	Producer entegrasyonu
Log4j2	(BOM)	Loglama çerçevesi + Kafka appender
OpenSearch	2.17.1	Log indeksleme ve sorgu
OpenSearch Java Client	2.14.0	Asenkron index erişimi

15.7. GÖZLEMLENEBİLİRLİK

Bileşen	Sürüm	Amaç
OpenTelemetry Collector	0.111.0	Trace toplama
Micrometer Tracing Bridge (OTel)	(BOM)	Spring → OTLP köprüsü
OpenTelemetry OTLP Exporter	(BOM)	OTLP/gRPC dışı aktarım
Micrometer Prometheus Registry	(BOM)	Uygulama metrikleri
Prometheus	v2.54.1	Metrik tarama (actuator/prometheus)
Grafana	11.2.0	Metrik dashboardları

15.8. API DOKÜMANTASYONU

Bileşen	Sürüm	Amaç
Springdoc-openapi	2.7.0	OpenAPI 3.1 üretimi, Swagger UI

15.9. ÇALIŞMA ORTAMI

Bileşen	Sürüm	Amaç
Docker Compose	—	Tüm servislerin orkestrasyonu
Eclipse Temurin JRE	21	Backend runtime imajı
Maven	3.9.9	Build (multi-stage Docker build)

15.10. TEST

Bileşen	Sürüm	Amaç
Spring Boot Test	(BOM)	Entegrasyon ve slice testleri
Spring Security Test	(BOM)	MockMvc + JWT Post-processor
JUnit 5	(BOM)	Test çatısı
Mockito	(BOM)	Test çiftleri (MockBean)

16. Sonuç

Bu doküman:

- Analiz dokümanındaki tüm kritik beklentileri kapsar
- Teknik davranışları açıkça tanımlar
- Akademik ve sektörel olarak savunulabilir

Bu doküman, analiz aşamasında tanımlanan iş beklentilerinin fazlandırılmış ve denetlenebilir teknik karşılığını sunar.

Doküman; akademik değerlendirme, sektörel uygulama ve yazılım geliştirme süreçleri için bağlayıcı bir referans niteliğindedir.